Security Review Report

Ain't all Risky Bizz Application

OWASP Top 10 Assessment

November 28, 2025

Executive Summary

- A comprehensive security review was performed based on the OWASP Top 10 (2021) standard.
- Key Findings:
- • 1 Critical Issue (Cryptographic Failures)
- • 1 High Issue (Security Misconfiguration)
- • 3 Medium Issues (Insecure Design, Components, Access Control)
- • Positive Results: No XSS or SQL Injection vulnerabilities found.

A02: Cryptographic Failures (CRITICAL)

- **Description:**
  - The application uses a hardcoded SECRET_KEY ('dev-secret-key') in the source code. This compromises all session security, allowing attackers to forge session cookies.
- **Evidence:**
  - app.py line 6: app.config['SECRET_KEY'] = 'dev-secret-key'
- **Recommendation:**
  - Use an environment variable for the secret key. Ensure it is a long, random string. Never commit secrets to version control.

A05: Security Misconfiguration (HIGH)

- **Description:**
  - Debug mode is enabled in the application configuration. This can leak sensitive stack traces, environment variables, and code snippets to attackers if an error occurs.
- **Evidence:**
  - app.py line 66: app.run(debug=True)
- **Recommendation:**
  - Disable debug mode in production. Use app.run(debug=False) or rely on a production WSGI server configuration.

A04: Insecure Design (MEDIUM)

- **Description:**
  - The application accepts POST requests (e.g., in the assessment wizard) without Anti-CSRF tokens. Attackers could trick users into submitting unauthorized data.
- **Evidence:**
  - HTML forms in assessment_wizard.html lack {{ csrf_token() }}.
- **Recommendation:**
  - Implement Flask-WTF or a similar library to generate and validate CSRF tokens for all state-changing forms.

A06: Vulnerable Components (MEDIUM)

- **Description:**
  - Dependency versions are not pinned in requirements.txt (e.g., 'flask' instead of 'flask==2.3.2'). This leads to non-reproducible builds and potential use of vulnerable versions.
- **Evidence:**
  - requirements.txt contains unpinned package names.
- **Recommendation:**
  - Pin exact versions in requirements.txt (e.g., Flask==3.0.0) and regularly scan dependencies for vulnerabilities.

A01: Broken Access Control (MEDIUM)

- **Description:**
  - No authentication or authorization mechanisms are implemented. The application is open to any user with network access.
- **Evidence:**
  - No @login_required decorators or user management logic in app.py.
- **Recommendation:**
  - Implement a user authentication system if the application is intended for restricted use. Ensure proper session management.

Positive Findings

- • A03: Injection (Pass)
- - Reflected XSS testing was unsuccessful (Jinja2 auto-escaping active).
- - SQL Injection risk is low due to SQLAlchemy ORM usage.
- • A10: SSRF (Pass)
- - No functionality found that fetches remote resources based on user input.

Recommended Next Steps

- 1. Immediate: Rotate the SECRET_KEY and use environment variables.
- 2. Immediate: Disable Debug Mode.
- 3. Short-term: Implement CSRF protection using Flask-WTF.
- 4. Short-term: Pin dependency versions in requirements.txt.
- 5. Long-term: Evaluate the need for user authentication.